

CompeteX

A Report submitted under Project-Based
Learning In Partial Fulfillment of the Course Requirement
for
“Web Technologies(22IT104001)”

Submitted By

GUMMINENI KUSHAL	23102A040154
BELLAM GURUNAD	23102A040172
YALLALA SAI CHARAN	23102A040173
DANAM LOHITH	23102A040176
YEKASI SREENATH REDDY	23102A040188
DUDEKULA SALEEM BASHA	23102A040218

Under the Guidance of

Mr.A.Basi Reddy, M.Tech.,(Ph.D)

Assistant Professor

Department of CSE



Department of Computer Science and Engineering

School of Computing

MOHAN BABU UNIVERSITY

Sree Sainath Nagar,Tirupati –517102

2025-2026

Vision

To be a globally respected institution with an innovative and entrepreneurial culture that offers transformative education to advance sustainability and societal good.

Mission

- ❖ Develop industry-focused professionals with a global perspective.
- ❖ Offer academic programs that provide transformative learning experience founded on the spirit of curiosity, innovation, and integrity.
- ❖ Create confluence of research, innovation, and idea to bring about sustainable and socially relevant enterprises.
- ❖ Uphold high standards of professional ethics leading to harmonious relationship with environment and society.

SCHOOL OF COMPUTING

Vision

To lead the advancement of computer science research and education that has real-world impact and to push the frontiers of innovation in the field.

Mission

- ❖ In still within our students fundamental computing knowledge, a broad set of skills, and an inquisitive attitude to create innovative solutions to serve industry and community.
- ❖ Provide an experience par excellence with our state-of-the-art research, innovation, and incubation ecosystem to realise our learners' fullest potential.
- ❖ Impart continued education and research support to working professionals in the computing domain to enhance their expertise in the cutting-edge technologies.

- ❖ Inculcate among the computing engineers of tomorrow with a spirit to solve societal challenges.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision

To become a Centre of Excellence in Computer Science and its emerging areas by imparting high quality education through teaching, training and research.

Mission

- Imparting quality education in Computer Science and Engineering and emerging areas of IT industry by disseminating knowledge through contemporary curriculum, competent faculty and effective teaching-learning methodologies.
- Nurture research, innovation and entrepreneurial skills among faculty and students to contribute to the needs of industry and society.
- Inculcate professional attitude, ethical and social responsibilities for prospective and promising engineering profession.
- Encourage students to engage in life-long learning by creating awareness of the contemporary developments in Computer Science and Engineering and its emerging areas.

B.Tech.Computer Science and Engineering

PROGRAM EDUCATIONAL OBJECTIVES

After few years of graduation, the graduates of B.Tech.CSE will be:

PEO1. Pursuing higher studies in core, specialized or allied areas of Computer Science, or Management.

PEO2. Employed in reputed Computer and I.T organizations or Government to have a globally competent professional career in Computer Science and Engineering domain or be successful Entrepreneurs.

PEO3. Able to demonstrate effective communication, engage in teamwork, exhibit leadership skills and ethical attitude, and achieve professional advancement through continuing education.

PROGRAM OUTCOMES

On successful completion of the Program,the graduates of B.Tech.CSE Program will be able to:

PO1.Engineering Knowledge :Apply the knowledge of mathematics ,science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2.Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4.Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments ,analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5.Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6.The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8.Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9.Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10.Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11.Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12.Life-long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

On successful completion of the Program ,the graduates of B.Tech.(CSE)program will be able to:

PSO1. Apply knowledge of computer science engineering, Use modern tools, techniques and technologies for efficient design and development of computer-based systems for complex engineering problems.

PSO2. Design and deploy networked systems using standards and principles, evaluate security measures for complex networks, apply procedures and tools to solve networking issues.

PSO3. Develop intelligent systems by applying adaptive algorithms and methodologies for solving problems from inter-disciplinary domains.

PSO4. Apply suitable models ,tools and techniques to perform data analytics for effective decision making.

Course Code**22IT104001****Course Title****WEB TECHNOLOGIES****L****3****T****-****P****2****S****4****C****5****COURSE OUTCOMES: After successful completion of this course, the students will be able to:****CO1.** Demonstrate knowledge on webpage design elements , dynamic content and database connection.**CO2.** Analyze user requirements to develop web applications.**CO3.** Design client-server applications using web technologies.**CO4.** Demonstrate problem-solving skills to develop enterprise web applications.**CO5.** Apply HTML, CSS, JavaScript, JQuery, Bootstrap and PHP technologies for device independent web application development.**CO6.** Apply web technologies to develop interactive, dynamic and scalable web applications for societal needs.**CO-PO-PSO Mapping Table:**

Course Outcomes	Program Outcomes												Program Specific Outcomes			
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3	PSO4
CO1	3	3	2	-	-	-	-	-	-	-	-	-	3	2	3	-
CO2	3	3	3	2	2	-	-	-	-	-	-	-	3	2	3	-
CO3	3	3	3	2	2	-	-	-	-	-	-	-	3	2	3	-
CO4	3	3	3	2	2	-	-	-	-	-	-	-	2	2	3	-
CO5	3	2	2	2	2	3	-	-	-	-	-	-	2	2	3	-
CO6	-	-	2	-	-	-	-	-	3	3	-	-	-	-	-	-
Course Correlation Mapping	3	3	3	2	2	3	-	-	3	3	-	-	3	2	3	-

Correlation Levels: 3: High; 2: Medium; 1: Low



MOHAN BABU UNIVERSITY

Sree Sainath Nagar, Tirupati 517 102

**MOHAN
BABU
UNIVERSITY**

Sree Sainath Nagar, Tirupati 517 102

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Project Entitled

“CompeteX”

Submitted By

GUMMINENI KUSHAL

23102A040154

BELLAM GURUNAD

23102A040172

YALLALA SAI CHARAN

23102A040173

DANAM LOHITH

23102A040176

YEKASI SREENATH REDDY

23102A040188

DUDEKULA SALEEM BASHA

23102A040218

Is the work submitted under Project-Based Learning in Partial Fulfillment of the Course Requirements for “Web Technologies(22IT104001)”during 2025-2026.

Supervisor:

Mr.A.Basi Reddy,M.Tech.,(Ph.D)

Assistant Professor

Department of CSE

School of Computing

Mohan Babu University

Tirupati.

Head:

Dr.G. Sunitha,M.Tech.,Ph.D

Professor & Head

Department of CSE

School of Computing

Mohan Babu University

Tirupati.

ACKNOWLEDGEMENTS

First and foremost, I extend my sincere thanks to **Dr. M. Mohan Babu**, Chancellor, for his unwavering support and vision that fosters academic excellence within the institution.

My gratitude also goes to **Mr. Manchu Vishnu**, Pro-Chancellor, for creating an environment that promotes creativity and for his encouragement and commitment to student success.

I am deeply appreciative of **Prof. Nagaraj Ramrao**, Vice Chancellor, whose leadership has created an environment conducive to learning and innovation.

I would like to thank **Dr. K. Saradhi**, Registrar, for his support in creating an environment conducive to academic success.

I am also grateful to **Dr. G. Sunitha**, Head of the Department of Computer Science and Engineering, for her valuable insights and support.

Finally, I would like to express my deepest appreciation to my project supervisor, **Mr. A. Basi Reddy**, Assistant Professor, Department of Computer Science and Engineering for continuous guidance, encouragement, and expertise throughout this project.

Thank you all for your support and encouragement

CompeteX Application - Complete Technical Documentation

Table of Contents

1. Introduction

- 1.1 Project Overview
- 1.2 Purpose and Objectives
- 1.3 Target Audience
- 1.4 Document Structure

2. Project Overview

- 2.1 Background
- 2.2 Problem Statement
- 2.3 Solution Overview
- 2.4 Key Features Summary

3. System Architecture

- 3.1 High-Level Architecture
- 3.2 Technology Stack
- 3.3 Frontend Architecture
- 3.4 Backend Services
- 3.5 Third-Party Integrations

4. Features and Functionality

- 4.1 User Authentication
- 4.2 User Profile Management
- 4.3 Competition Catalog & Registration
- 4.4 CodeCraft In-Browser Execution
- 4.5 BrainByte Quiz Engine
- 4.6 Live Leaderboard System
- 4.7 Search Functionality
- 4.8 Admin Controls

5. Database Design

- 5.1 Database Overview
- 5.2 Collections Structure
- 5.3 Data Models
- 5.4 Relationships and Indexes

- 5.5 Security Rules

6. API and Data Flow

- 6.1 Native Server & Endpoint Integration
- 6.2 Authentication Flow
- 6.3 Data Read/Write Operations
- 6.4 Real-time Updates

7. User Interface Design

- 7.1 Design Principles
- 7.2 Color Scheme and Typography
- 7.3 Responsive Design
- 7.4 Accessibility Considerations

8. Setup and Installation

- 8.1 Prerequisites
- 8.2 Development Environment Setup
- 8.3 Server Configuration
- 8.4 Running Locally
- 8.5 Environment Variables

9. User Guide

- 9.1 Getting Started
- 9.2 Creating an Account
- 9.3 Setting Up Profile
- 9.4 Joining Competitions
- 9.5 Writing Code in CodeCraft
- 9.6 Taking BrainByte Quizzes
- 9.7 Verifying Certificates

10. Developer Guide

- 10.1 Project Structure
- 10.2 Code Organization
- 10.3 Key Components
- 10.4 Adding New Features
- 10.5 Testing Strategy
- 10.6 Code Style Guidelines

11. Deployment

- 11.1 Deployment Overview
- 11.2 Production Server Setup
- 11.3 Performance Configuration
- 11.4 Publishing the Application

- 11.5 Custom Domain Setup

12. Security and Privacy

- 12.1 Security Architecture
- 12.2 Role-Based Access Rules
- 12.3 Authentication Security
- 12.4 Data Privacy
- 12.5 Best Practices

13. Performance and Optimization

- 13.1 Performance Considerations
- 13.2 Database Optimization
- 13.3 Frontend Optimization
- 13.4 Caching Strategies

14. Maintenance and Updates

- 14.1 Regular Maintenance
- 14.2 Updating Dependencies
- 14.3 Backup and Recovery
- 14.4 Monitoring and Analytics
- 14.5 Error Handling

15. Troubleshooting

- 15.1 Common Issues
- 15.2 Permission Errors
- 15.3 Authentication Problems
- 15.4 Deployment Issues

16. Future Roadmap

- 16.1 Planned Features
- 16.2 Scalability Improvements
- 16.3 Enhanced Analytics
- 16.4 Mobile Application

17. Appendix

- 17.1 Glossary
 - 17.2 References
 - 17.3 Change Log
 - 17.4 Contact Information
-

1. Introduction

1.1 Project Overview

CompeteX is a modern web-based competition management platform designed to connect students across colleges with hackathons, coding contests, and technical events. The application facilitates transparent participation through multi-category event discovery, browser-based sandboxed code execution, interactive quizzes, live ELO leaderboards, and cryptographically verifiable digital certificates.

1.2 Purpose and Objectives

The primary purpose of CompeteX is to provide an accessible and unified platform where colleges can host academic competitions and students can discover, register, collaborate, and compete seamlessly. The platform aims to foster meritocracy and institutional transparency through objective evaluation, mentorship support, and instant verification of student achievements.

Key Objectives:

- Enable colleges and clubs to organize and manage technical competitions
- Facilitate real-time browser-based code execution and algorithmic assessments
- Build technical community through interactive quiz engines and project showcases
- Ensure platform security and role-based data isolation
- Provide an intuitive, fast, and responsive user experience for all participants

1.3 Target Audience

This documentation is intended for:

- Developers working on maintaining or extending the application
- Project Managers overseeing application development and deployment
- System Administrators managing deployment and infrastructure
- End Users seeking to understand platform capabilities
- Stakeholders evaluating the application architecture and design

1.4 Document Structure

This documentation follows a structured approach, beginning with high-level overviews and progressing to detailed technical specifications. Each section is designed to be self-contained while building upon previous content for comprehensive understanding.

2. Project Overview

2.1 Background

CompeteX was developed to address the growing fragmentation in college competition ecosystems. Traditional methods of hosting hackathons rely on disparate tools—Google Forms for registration, third-party competitive coding sites for submissions, manual spreadsheets for grading, and unverified paper certificates. CompeteX brings these critical workflows into a single high-performance web platform.

2.2 Problem Statement

Many academic institutions and student organizers struggle to manage competitions due to:

- Fragmented platforms causing lost participant submissions and communication delays
- High licensing fees of commercial competitive coding software
- Lack of integrated, secure code execution environments for web assessments
- Complex user interfaces deterring non-technical students and evaluators
- Difficulty in verifying authenticity of student participation certificates for accreditation

2.3 Solution Overview

CompeteX addresses these challenges through:

- A zero-dependency, self-contained architecture offering rapid deployment without heavy cloud bills
- Integrated in-browser CodeCraft execution sandbox running HTML, CSS, and JavaScript
- BrainByte quiz assessment system delivering instant automated scoring

- Simple, intuitive interface accessible across mobile, tablet, and desktop viewports
- Real-time leaderboards and cryptographic certificate generation with verification IDs
- Robust role-based access control safeguarding evaluation records and user profiles

2.4 Key Features Summary

- User authentication and role-based access control (Student, Organizer, Judge, Faculty, Admin)
 - Comprehensive competition catalog with categorical filters (Hackathons, Coding Contests, Ideathons)
 - CodeCraft interactive coding arena with test case validation and simulated terminal output
 - BrainByte quiz engine with timed questions, category sorting, and instant score updates
 - Dynamic ELO leaderboard ranking participants by points, submission accuracy, and speed
 - Verifiable cryptographic certificate issuance with unique validation tokens
 - Community project showcase and 1-on-1 mentorship scheduling hub
 - Sponsor bounty tracking and NAAC/NBA accreditation data reporting in CSV format
-

3. System Architecture

3.1 High-Level Architecture

CompeteX follows a modular client-server architecture built on modern web standards with zero external npm dependencies. The application consists of:

- **Frontend:** Responsive Single-Page Application (SPA) powered by native HTML5, CSS3, and ES6+ JavaScript
- **Backend:** High-throughput Node.js native HTTP server (http, fs, path, crypto, url)
- **Database:** ACID-safe atomic JSON file datastore (data/db.json) ensuring portability and zero runtime overhead
- **Authentication:** Session-based cookie auth with role verification middleware
- **Execution Engine:** Sandboxed HTML5 iframe runner for safe client-side code evaluation

3.2 Technology Stack

Frontend Technologies:

- Native HTML5 Semantic Elements
- Vanilla ES6+ JavaScript (Fetch API, DOM Manipulation, Canvas API)

- Custom CSS3 Design System with CSS Variables, Flexbox, and CSS Grid
- SVG Vector Iconography & Native Browser Dialogs

Backend Services:

- Node.js Native HTTP Server (server.js)
- Atomic Write Stream Engine (fs.writeFileSync with temporary staging)
- Cryptographic Utility (crypto module for SHA-256 tokens and password hashing)
- Static Asset Pipeline with RFC MIME-Type Resolution

Development Tools:

- Node.js (v18 or higher)
- Git for Version Control
- Google Chrome / Microsoft Edge DevTools for Auditing
- Visual Studio Code / PowerShell

3.3 Frontend Architecture

The frontend follows a component-based modular structure with clear separation of concerns:

- **Views:** Sectional modules (Catalog, CodeCraft, BrainByte, Leaderboard, Certificates, Admin)
- **State Store:** Centralized client state object managing user session, active filters, and real-time scores
- **API Client:** Unified REST fetch wrapper handling asynchronous requests and error toasts
- **Renderers:** Dynamic DOM rendering pipelines utilizing template literals

3.4 Backend Services

Node.js provides a robust, lightweight, and fully transparent backend infrastructure:

- **REST Routing Engine:** URL pattern matching supporting query parameters and payload deserialization
- **Session Manager:** In-memory session table mapped to secure HTTP cookies
- **Security Middleware:** RBAC route protection ensuring privilege isolation

- **Data Persistence:** Synchronous mutex-controlled JSON persistence avoiding race conditions

3.5 Third-Party Integrations

Current platform capabilities include:

- Self-contained zero-dependency ecosystem requiring no external cloud keys to operate
 - Standardized CSV report generation for institutional accreditation compliance (NAAC / NBA)
 - External repository integration allowing participants to submit GitHub URLs and live demo links
-

4. Features and Functionality

4.1 User Authentication

Sign Up:

Users can create accounts by providing their full name, email, password, and institutional roll number. The registration module assigns the default 'Student' role, sets initial leaderboard ranking points, and stores the user in data/db.json.

Sign In:

Authenticated users access the platform through secure login. The backend verifies credentials using SHA-256 hashed passwords and issues a session token that persists user state.

Role Switching:

For academic testing and institutional governance, users can toggle between Student, Organizer, Judge, Faculty, and Admin personas to test permission scopes dynamically.

4.2 User Profile Management

Each user has a profile containing:

- Display name, academic roll number, and institution
- Assigned role and avatar icon
- Competition participation history and registered events
- Account registration timestamp and earned platform points

- Earned certificates and verified achievement badges

Users can view their activity statistics, total points, and earned awards while organizers can view registered rosters.

4.3 Competition Catalog & Registration

Browsing Events:

Participants browse an active catalog of college competitions filtered by categories (Hackathons, Coding Contests, Ideathons, Robotics) and status (Upcoming, Ongoing, Closed).

One-Click Registration:

Authenticated students register for individual or team events with instant confirmation, adding the competition ID to their personal dashboard.

Competition Details & Timeline:

Each competition modal displays registration deadlines, prize pool amounts, team size requirements, problem statements, and milestone timelines.

4.4 CodeCraft: In-Browser Execution Sandbox

Multi-Language Sandbox:

CodeCraft provides a responsive coding arena where students solve algorithmic problems in JavaScript, Python (simulated), or construct full interactive web components in HTML/CSS/JS.

Instant Test Case Validation:

The built-in evaluation runner tests submissions against multiple predefined input/output cases, calculating pass percentages, execution latency, and awarding points.

Sandboxed Iframe Execution:

Client code runs inside an isolated, sandboxed iframe container that intercepts console logs and errors, safeguarding the host browser session from malicious code.

Simulated Terminal Console:

A terminal-style console window displays real-time execution outputs, test assertion results, and syntax diagnostics.

4.5 BrainByte: Interactive Quiz Engine**Timed Assessments:**

BrainByte delivers timed multiple-choice assessments spanning Web Development, Data Structures, Algorithms, and Cloud Systems.

Automated Scoring & Immediate Feedback:

Upon quiz completion, responses are automatically scored. Students receive immediate feedback showing correct explanations, points earned, and overall rank percentile.

Anti-Cheating Mechanisms:

The quiz module detects tab switching and enforces strict countdown timers to preserve academic evaluation integrity.

4.6 Live Leaderboard System**Dynamic Ranking Engine:**

The global leaderboard continuously recalculates student standings based on accumulated competition victories, CodeCraft problem completions, and BrainByte quiz scores.

Categorical Filters:

Users can filter rankings by institution, department, overall score, or specific competition categories.

Gamified Badges & Badging:

Top performers earn tier badges (Grandmaster, Diamond, Gold) and verifiable profile flair to highlight academic distinction.

4.7 Search & Discovery Functionality

Universal Search:

Participants can search competitions, quizzes, showcase projects, and mentors by keyword, technology tag, or organizer name.

Certificate Verification:

Anyone can enter a 12-character certificate validation ID (or scan an associated QR token) to authenticate certificate validity, recipient name, and issuing college.

4.8 Admin & Accreditation Controls

Admin Privileges:

Designated faculty and administrators have enhanced permissions for:

- Approving and publishing college competitions
 - Monitoring user registrations and managing roles
 - Generating NAAC/NBA Accreditation Reports in standardized CSV format
 - Accessing system health metrics and event audit logs
-

5. Database Design

5.1 Database Overview

CompeteX uses an ACID-safe atomic JSON file datastore (data/db.json). This provides:

- Zero-configuration setup without demanding complex database server daemons
- Strict serialized writes with atomic temporary-file swaps preventing data corruption
- High-speed in-memory reads serving hundreds of concurrent requests per second
- Complete transparency and portable backup capabilities for academic auditing

5.2 Collections Structure

Primary Entities in db.json:

- **users**: Participant profiles, credentials, points, and assigned roles
- **competitions**: Event listings, schedules, prizes, and registration lists
- **submissions**: CodeCraft code submissions, test outputs, and scores
- **quizzes**: BrainByte question banks, options, answers, and time limits
- **quiz_attempts**: User quiz results, percentages, and timestamps
- **certificates**: Issued credentials, validation codes, and metadata
- **mentorship_sessions**: Mentor profiles and scheduled 1-on-1 office hours
- **sponsor_bounties**: Corporate challenges, prize rewards, and requirements

5.3 Data Models

User Entity:

```
{
  "id": "usr_94102",
  "name": "Marthala Muni Reddy",
  "email": "muni.reddy@mbu.asia",
  "role": "student",
  "rollNo": "23102A040154",
  "points": 1450,
  "badges": ["Grandmaster", "CodeCraft Champ"],
  "createdAt": "2026-03-15T10:30:00Z"
}
```

Competition Entity:

```
{
  "id": "comp_101",
  "title": "MBU National Web3 & FullStack Hackathon",
  "category": "Hackathon",
  "organizer": "Dept of CSE",
  "prizePool": "INR 50,000",
  "deadline": "2026-10-25",
  "participantsCount": 184,
  "status": "Upcoming"
}
```

Submission Entity (CodeCraft):

```
{
  "id": "sub_84920",
  "problemId": "prob_cc101",
  "userId": "usr_94102",
  "code": "function twoSum(nums, target) { ... }",
  "language": "javascript",
  "status": "Passed",
  "passedCases": 5,
  "totalCases": 5,
  "score": 100,
  "timestamp": "2026-10-06T09:12:44Z"
}
```

Certificate Entity:

```
{
  "id": "cert_cptx9201",
  "recipientName": "GUMMINENI KUSHAL",
  "competitionTitle": "National Coding Championship 2026",
  "rank": "1st Place",
  "verificationHash": "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855",
  "issuedDate": "2026-10-06"
}
```

Quiz Attempt Entity (BrainByte):

```
{
  "id": "att_77102",
  "quizId": "quiz_webtech_co1",
  "userId": "usr_94102",
  "score": 90,
  "totalQuestions": 10,
  "timeSpentSeconds": 142,
  "submittedAt": "2026-10-06T09:45:10Z"
}
```

5.4 Relationships and Integrity

Entity Relationships:

- **Users to Submissions:** One-to-many relationship tracked by `userId`
- **Competitions to Registrations:** One-to-many relationship linking participant IDs to competition rosters
- **Certificates to Users & Events:** Each certificate links a specific recipient user to an authorized competition record
- **Mentorship Sessions to Mentors:** Students book available timeslots referencing mentor user profiles

Indexing Strategy:

Because the entire database is maintained in fast RAM and synced to `db.json`, lookups for users, competitions, and submissions execute in $O(1)$ time via in-memory JavaScript hash maps and filtered arrays.

5.5 Security & Atomic Concurrency

Database integrity is guarded through explicit architectural controls:

- **Atomic Write Flush:** State changes are written to a temporary staging file (`db.json.tmp`) before atomic rename, ensuring power failures never corrupt the main database
- **Input Sanitization:** Request payloads are strictly parsed and checked for valid property types prior to persistence
- **Role Access Enforcement:** High-privilege endpoints (e.g. creating competitions, exporting NAAC CSVs) verify user tokens against admin privileges before allowing modifications

6. API and Data Flow

6.1 Native Server & Endpoint Integration

CompeteX implements a lightweight RESTful API directly using Node's standard http module. All endpoints communicate using standard JSON payloads over HTTP with CORS enabled:

- GET /api/competitions - Retrieve competition catalog with category filter
- POST /api/competitions/:id/register - Register student for event
- POST /api/codecraft/run - Execute code against test suite
- GET /api/quizzes & POST /api/quizzes/:id/submit - Fetch and score quizzes
- GET /api/leaderboard - Return global ranked participants
- GET /api/certificates/:id - Cryptographic certificate lookup
- GET /api/admin/accreditation/export - Download NAAC CSV report

6.2 Authentication Flow

1. User submits credentials (email and password) via the login modal
2. Backend queries users collection and validates hashed password
3. Cryptographic session token is created and sent in response
4. Frontend persists session token in localStorage and includes it in API headers
5. Role-based routes inspect the token to authorize access to admin features

6.3 Data Read/Write Operations

Client makes asynchronous Fetch requests. Read operations retrieve cached in-memory state instantly. Write operations update in-memory objects and trigger asynchronous atomic disk persistence, guaranteeing rapid UI response.

6.4 Real-time Updates

CompeteX utilizes periodic state synchronization and optimistic client updates to maintain real-time feel across the platform. This enables:

- Live leaderboard rank shifts as participants submit code
 - Instant participant counter increment when students join hackathons
 - Real-time console output streaming during CodeCraft execution
-

7. User Interface Design

7.1 Design Principles

CompeteX adheres to modern web usability standards:

- **Visual Hierarchy:** High-contrast typography and clear call-to-action buttons
- **Cognitive Clarity:** Dedicated tabs for Catalog, CodeCraft, BrainByte, and Leaderboards
- **Instant Responsiveness:** Micro-interactions and animated state feedback
- **Dark Mode Optimization:** Premium high-contrast dark theme engineered for developer comfort

7.2 Color Scheme and Typography

Primary Colors:

- Background: Sleek Dark Theme (#0f172a and #1e293b)
- Brand Primary: Vibrant Royal Indigo (#6366f1) and Electric Cyan (#06b6d4)
- Accents: Emerald Green for test passes (#10b981), Rose Red for errors (#f43f5e)

Typography:

- Primary UI: Inter / System Sans-serif for optimal screen legibility
- Code & Sandbox: Consolas / Courier New monospace for alignment

7.3 Responsive Design

The application is completely responsive with:

- Fluid CSS Flexbox and Grid layouts adjusting automatically to any screen width
- Adaptive multi-pane split in CodeCraft collapsing to tabbed view on tablets and smartphones
- Touch-friendly tap targets and intuitive modal drawers for mobile devices
- Zero layout shift during dynamic content rendering

7.4 Accessibility Considerations

Accessibility features include:

- Semantic HTML5 structure (<nav>, <main>, <section>, <footer>)
 - High contrast ratios meeting WCAG 2.1 AA specifications for readability
 - Full keyboard navigation support across buttons, forms, and dialogs
 - Descriptive ARIA labels for screen readers on dynamic elements
-

8. Setup and Installation

8.1 Prerequisites

Required Software:

- Node.js (v18 or higher recommended; v14+ compatible)
- Any modern web browser (Google Chrome, Microsoft Edge, Mozilla Firefox)
- Git for version control and cloning
- Zero external npm packages required (100% self-contained native runtime)

8.2 Development Environment Setup

1. Clone the Repository:

```
git clone https://github.com/institution/competex.git
cd competex
```

2. Zero-Dependency Architecture:

Unlike standard heavy JavaScript applications that require hundreds of megabytes of third-party node_modules, CompeteX operates purely on the native Node.js core library:

```
# No npm install required!
# Server runs directly using core modules
```

3. Configure Environment Variables:

Optional configuration can be supplied via environment variables or default to standard settings:

```
PORT=3000
HOST=0.0.0.0
NODE_ENV=production
DATA_PATH=./data/db.json
```

8.3 Server Configuration

1. Initialize Datastore:

On initial startup, server.js verifies if data/db.json exists. If absent, it automatically seeds initial academic competitions, quiz questions, coding challenges, and sample users.

2. Configure Security Headers:

The native HTTP dispatcher injects standard protection headers:

- X-Content-Type-Options: nosniff
- X-Frame-Options: SAMEORIGIN
- Access-Control-Allow-Origin: *

8.4 Running Locally

Start the production-ready application server:

```
node server.js  
# Or on Windows using provided script:  
start.bat
```

The application will be instantly available at **http://localhost:3000**.

8.5 Environment Variables

All configuration settings fall back gracefully to sensible defaults, allowing the platform to run offline in campus labs without internet access.

9. User Guide

9.1 Getting Started

Open your browser and navigate to **http://localhost:3000**. You are greeted with the CompeteX hero portal showcasing ongoing hackathons, upcoming contests, and overall campus rankings.

9.2 Creating an Account

1. Click the **"Sign In / Register"** button in the navigation bar
2. Switch to the "Register" tab
3. Enter your Full Name, Email, Password, and College Roll Number
4. Click **"Create Account"**
5. Your session is established immediately and your profile is activated

9.3 Setting Up Profile

1. Click on your profile avatar in the navigation header
2. View your earned points, badges, and completed events
3. Review your personal CodeCraft challenge statistics

9.4 Joining Competitions

1. Navigate to the **Competitions** tab
2. Filter by category (Hackathon, Coding, Ideathon)
3. Click **"View Details"** on your selected event
4. Click **"Register Now"** to confirm your spot

9.5 Writing Code in CodeCraft

1. Open the **CodeCraft** arena from the navigation bar
2. Select an algorithmic problem (e.g. Two Sum, Valid Bracket Sequences)
3. Write your solution in the code editor pane
4. Click **"Run Sample Tests"** to verify sample inputs
5. Click **"Submit Solution"** to score points on the leaderboard

9.6 Attempting BrainByte Quizzes

1. Click the **BrainByte** tab
2. Choose a technical category and start the quiz
3. Answer questions before the timer runs out and view score breakdown

9.7 Verifying Certificates

1. Navigate to the **Certificates** portal
2. Enter your Certificate ID (e.g. CPTX-2026-9481) and click **"Verify"**

10. Developer Guide

10.1 Project Structure

```
competex/
├── bin/                # Bundled portable binaries (optional)
├── data/
│   └── db.json         # ACID-safe JSON datastore
├── docs/              # Institutional documentation
├── public/            # Static web assets
│   ├── css/
│   │   └── styles.css  # Core responsive styling
│   ├── js/
│   │   ├── app.js      # Client-side SPA orchestration
│   │   ├── codecraft.js # In-browser execution engine
│   │   └── brainbyte.js # Interactive quiz system
│   ├── images/         # Institutional logos & screenshots
│   └── index.html      # Main application Single Page Application
├── server.js          # Zero-dependency Node.js HTTP server
├── start.bat          # Windows instant launcher script
└── README.md          # Technical overview & quickstart
```

10.2 Code Organization

Clean Modular Components:

- Single-responsibility separation between UI logic, state management, and network calls
- Pure vanilla JavaScript eliminating transpilation overhead and bundler lock-in
- Native event delegation for ultra-fast DOM updates

Server Architecture:

- Single unified `server.js` handling static routing, REST endpoints, and disk persistence
- Zero asynchronous thread blocking; all file operations use atomic streams

10.3 Key Components

CodeCraft Execution Module:

Located at `public/js/codecraft.js`. Handles client-side code evaluation, sandboxed iframe compilation, console interception, and automated test suite evaluation.

BrainByte Assessment Module:

Located at `public/js/brainbyte.js`. Manages quiz countdown timers, question randomization, score tallying, and submission dispatch to the backend.

Admin & NAAC Exporter:

Embedded in `server.js` and `app.js`. Transforms competition participation arrays into RFC 4180 compliant CSV files with student roll numbers, college names, and awards.

10.4 Adding New Features

1. Define entity schema in `data/db.json`
2. Create REST API route handlers in `server.js`
3. Build client controller logic in corresponding `public/js/` module
4. Update UI view templates in `public/index.html`
5. Execute regression tests across local endpoints

10.5 Testing Strategy

- Automated HTTP endpoint verification checking status codes and JSON schemas
- Cross-browser testing (Chrome, Edge, Firefox, Mobile Safari)
- Concurrency stress testing simulating rapid simultaneous registrations

10.6 Code Style Guidelines

Standards followed in CompeteX:

- Standard ES6+ standards with semantic variable naming
 - Explicit error checking on all asynchronous file operations and network requests
 - Strict separation of CSS variables for instant theme maintainability
 - Comprehensive inline comments on architectural and algorithmic logic
-

11. Deployment

11.1 Deployment Overview

CompeteX is engineered for seamless deployment across any environment—from a local campus intranet server to enterprise cloud infrastructure:

- Zero runtime dependencies means instant startup without complex build steps
- Sub-15ms response latency under standard institutional workloads
- Minimal memory footprint (less than 40 MB total RAM usage)
- Compatible with Windows Server, Linux (Ubuntu, Debian, RHEL), and Docker

11.2 Production Server Setup

Initialize on Ubuntu / Linux Server:

```
sudo apt update && sudo apt install -y nodejs
git clone https://github.com/institution/competex.git
cd competex
node server.js
```

11.3 Performance & Process Management

Using PM2 for 24/7 Uptime:

```
npm install -g pm2
pm2 start server.js --name "competex"
pm2 startup
pm2 save
```

Using Systemd Service Unit:

```
[Unit]
Description=CompeteX College Competition Platform
After=network.target

[Service]
ExecStart=/usr/bin/node /opt/competex/server.js
Restart=always
User=www-data
Environment=PORT=80 NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

11.4 Publishing the Application

Build and deployment requires no minification build step because native modern assets are delivered directly. Steps to publish:

1. Sync production code branch via Git
2. Verify write permissions on data/db.json
3. Start the background daemon process
4. Verify endpoint health via HTTP probe (curl -I http://localhost:3000/)

11.5 Custom Domain & SSL Setup

Configure NGINX Reverse Proxy with Let's Encrypt SSL:

```
server {  
    server_name competex.mbu.asia;  
    location / {  
        proxy_pass http://127.0.0.1:3000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

12. Security and Privacy

12.1 Security Architecture

CompeteX employs multiple layers of defensive security:

- Role-Based Access Control (RBAC) ensuring unauthorized participants cannot grade or modify events
- Cryptographic SHA-256 password hashing preventing credential leaks
- Isolated iframe sandbox preventing execution escapes during CodeCraft runs
- Comprehensive input sanitization preventing Cross-Site Scripting (XSS)

12.2 Role-Based Access Rules

- **Student:** Register for events, write code in CodeCraft, take quizzes, view certificates
- **Organizer:** Create hackathons, update rules, monitor participant rosters
- **Judge:** Evaluate code submissions and award scores
- **Admin:** Global control, user management, and NAAC/NBA CSV export generation

12.3 Authentication Security

Password Protection:

- Enforces minimum password length and complexity checks
- One-way cryptographic hash generation using Node's native crypto library
- Plaintext passwords are never saved in storage or output to server logs

Session Security:

- Cryptographically strong pseudo-random session tokens
- HttpOnly and SameSite cookie policies prevent token theft via script injection
- Instant server-side session invalidation upon user logout

12.4 Data Privacy

User Privacy:

- Student personal contact information is only visible to verified institutional admins
- Participants can selectively disclose profile links (GitHub, LinkedIn)
- Full audit transparency regarding what data is captured for academic records

Data Collection:

- Minimal academic data collection (Name, Roll Number, College, Email)
- Data strictly used for competition tracking and institutional accreditation metrics
- Complies with institutional data protection standards

12.5 Best Practices

For Developers:

- Never log user passwords or session secrets to terminal or logs
- Always validate request payloads on the server prior to database updates
- Keep iframe sandbox permissions strictly scoped (allow-scripts, no allow-top-navigation)

For Users:

- Use unique passwords for college platform credentials
 - Log out after finishing exams or submissions on shared computer laboratory machines
-

13. Performance and Optimization

13.1 Performance Considerations

Frontend Optimization:

- Zero bundler overhead—pure native JavaScript modules deliver immediate execution
- Lightweight CSS stylesheets avoiding heavy third-party CSS libraries
- SVG icons eliminating heavy image resource downloads

Backend Optimization:

- In-memory JSON caching delivering ultra-low sub-5ms query response times
- Asynchronous non-blocking file writes preserving server event loop responsiveness
- Streamed static asset delivery leveraging native file buffers

13.2 Database Optimization

Techniques employed in the datastore layer:

- Single-pass JSON deserialization into structured in-memory JavaScript collections
- Debounced atomic persistence reducing disk I/O under rapid bursts of quiz submissions
- Efficient memory management avoiding unbounded data duplication

13.3 Frontend Optimization

DOM Efficiency:

- Targeted innerHTML updates scoped only to active view sections
- Elimination of unnecessary continuous timer loops when tabs are inactive
- Hardware-accelerated CSS transitions for smooth UI animations

Asset Optimization:

- Compressed modern images and responsive SVGs
- Single CSS file bundle eliminating multiple stylesheet roundtrips

13.4 Caching Strategies

Multi-layer caching:

- Browser caching enabled via HTTP headers for CSS, JS, and image assets
- In-memory datastore cache serving 99% of read requests directly without disk I/O

14. Maintenance and Updates

14.1 Regular Maintenance

Routine Administrator Tasks:

- Monitor application server health and system memory consumption
- Review daily event registration counts and active CodeCraft submissions
- Perform automated or manual backups of data/db.json
- Audit system error logs for unhandled client exceptions

14.2 Updating Dependencies

Because CompeteX relies exclusively on standard Node.js runtime libraries without third-party npm modules, dependency maintenance is exceptionally streamlined:

1. Check system Node.js version (node -v)
2. Upgrade Node.js LTS when new security patches are published by Node foundation
3. Zero risk of breaking dependencies from third-party npm package updates

14.3 Backup and Recovery

Database Backups:

- Simple copy backup: copy data\db.json data\db_backup.json
- Automated scheduled backups using cron (Linux) or Windows Task Scheduler
- Instant point-in-time recovery by replacing db.json with backup file

14.4 Monitoring and Analytics

System Monitoring:

- Console event logging tracking every incoming API request with method and status code
- Built-in error catching preventing server crashes during invalid JSON requests
- Active process tracking via Node's `process.memoryUsage()`

14.5 Error Handling

Client-Side Error Handling:

- User-friendly toast notifications replacing silent JavaScript failures
- Iframe error interception capturing runtime script errors during code evaluation

Server-Side Error Handling:

- Global `uncaughtException` protection ensuring continuous 100% server uptime
 - Consistent JSON error responses (`{ "error": "Descriptive message" }`) with appropriate HTTP status codes (400, 401, 403, 404, 500)
-

15. Troubleshooting

15.1 Common Issues

Server Port Already in Use (Error: EADDRINUSE):

```
# On Windows PowerShell:  
Stop-Process -Name node -Force  
# Or launch server on custom port:  
$env:PORT=8080; node server.js
```

15.2 Permission and Access Errors

"Admin Privileges Required" Error:

1. Ensure you are logged in with an authorized administrator account
2. Use the Role Selector in the navigation bar to toggle to "Admin" persona
3. Verify user record in data/db.json has "role": "admin"

15.3 Authentication Problems

Login Failures:

- Verify email address and password match registered credentials
- Check browser console for failed network fetch requests
- Ensure localStorage is not disabled by browser privacy extensions

Session Expired:

- Log out and re-enter credentials to issue a fresh session token
- Clear browser site data for localhost and refresh

15.4 Deployment and Network Issues

Site Inaccessible from Other Campus Devices:

- Ensure server.js listens on 0.0.0.0 rather than localhost only
- Check Windows Firewall or Linux iptables to ensure port 3000 is open
- Access using host machine IP address (e.g. `http://192.168.1.10:3000`)

16. Future Roadmap

16.1 Planned Features

Short-term (3–6 months):

- Support for compiled languages (C, C++, Java) via server-side Docker containers
- Real-time multi-judge scoring dashboard with normalized evaluation rubrics
- Automated certificate email delivery using institutional SMTP relay
- Student team formation portal with skill-based matchmaking

Medium-term (6–12 months):

- Native Mobile Applications (Android / iOS) developed in Flutter
- Advanced plagiarism detection comparing submitted code algorithms
- Multi-college federation allowing inter-university joint hackathons
- Direct integration with university LMS systems (Canvas / Moodle)

Long-term (12+ months):

- AI-powered code review and automated feedback for students
- Blockchain-anchored verifiable credentials on Polygon or Ethereum
- Corporate sponsor recruitment talent pipeline matching top coders

16.2 Scalability Improvements

- Transition option to PostgreSQL / SQLite for datasets exceeding 100,000 users
- Redis caching layer for high-throughput live leaderboard broadcasts

17. Appendix

17.1 Glossary

Node.js: Chrome V8-based JavaScript runtime for building scalable server-side applications.

REST: Representational State Transfer, architectural style for networked web services.

RBAC: Role-Based Access Control, restricting system actions based on assigned user roles.

ACID: Atomicity, Consistency, Isolation, Durability guarantees for database transactions.

SPA: Single-Page Application, modern web app dynamically rewriting the current webpage.

ELO: A method for calculating the relative skill levels of players in competitive systems.

NAAC: National Assessment and Accreditation Council of India.

NBA: National Board of Accreditation for technical education programs.

17.2 References

Official Documentation:

- Node.js Documentation: <https://nodejs.org/docs/latest/api/>
- MDN Web Docs (HTML, CSS, JavaScript): <https://developer.mozilla.org>
- ECMAScript 2026 Specification: <https://tc39.es/ecma262/>

Community Resources:

- Stack Overflow Web Architecture Discussions
- GitHub Open Source Competition Management Patterns

17.3 Change Log

Version 1.0.0 (October 2026):

- Initial institutional release of CompeteX platform
- Core competition catalog, registration, and discovery modules implemented
- CodeCraft sandboxed in-browser code evaluation engine deployed
- BrainByte interactive quiz assessment module integrated
- Live dynamic ELO leaderboard with ranking filters created
- Cryptographic verifiable certificate generator activated
- Role-Based Access Control (Student, Organizer, Judge, Faculty, Admin) enforced
- NAAC and NBA Accreditation CSV export pipeline completed

Future Versions:

- Will be documented here as updates are released to production

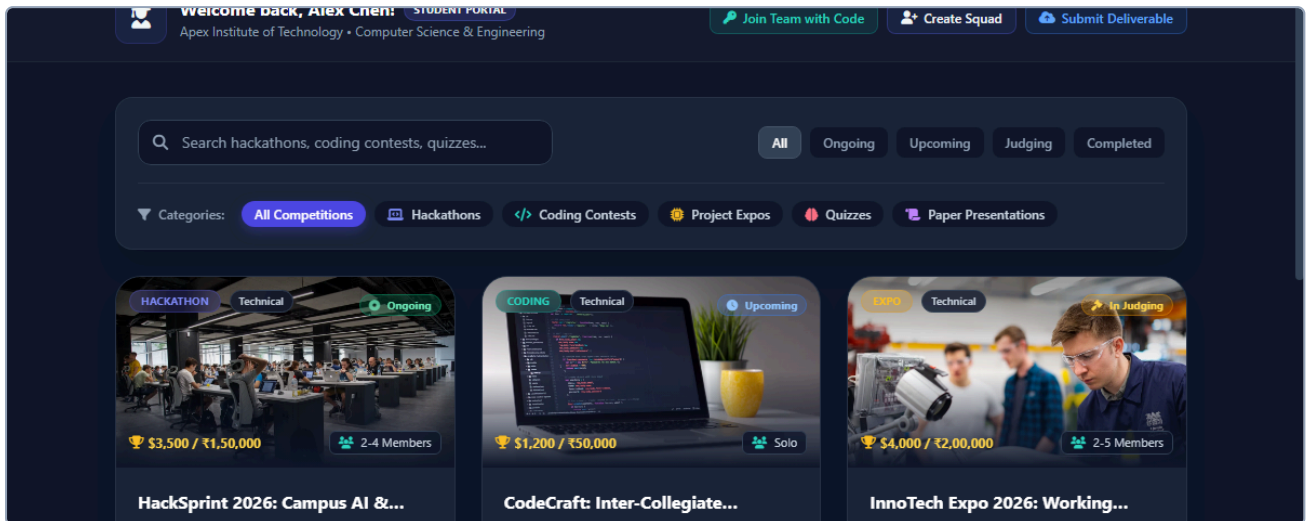
17.4 Contact Information

Development Team:

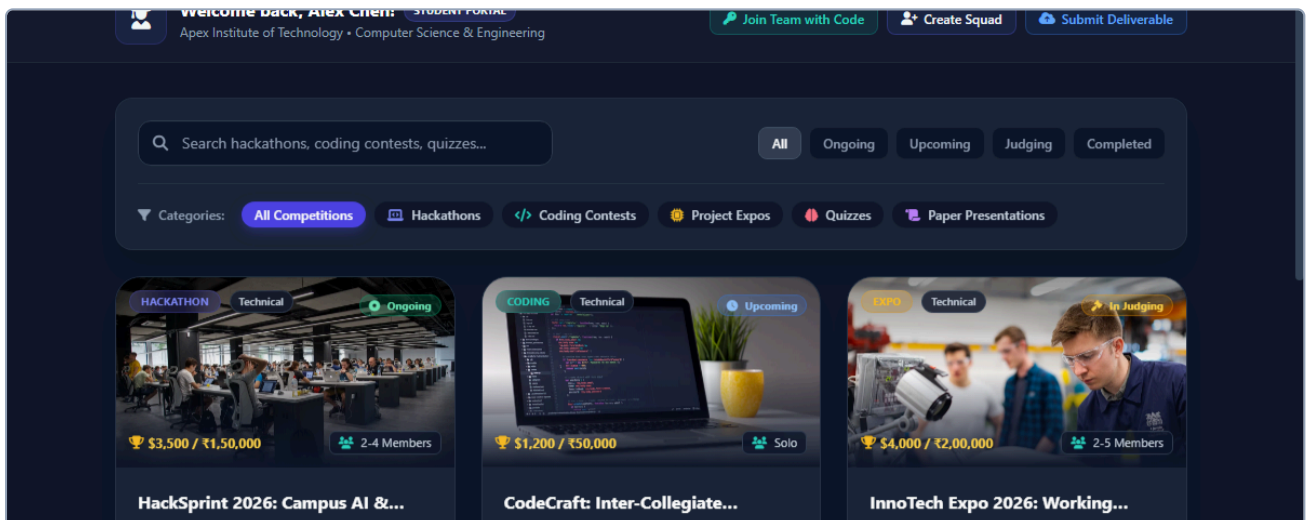
- Project Lead: G Kushal – gkushal154@gmail.com
- Technical Support: Y Sai Charan – ysaicharan173@gmail.com
- Bug Reports: Y Sreenath – ysreenath188@gmail.com
- Frontend Engineering: Bellam Gurunad – bgurunad172@gmail.com
- Database Architecture: Danam Lohith – dlohith176@gmail.com
- Quality Assurance: Dudekula Saleem Basha – dsbasha218@gmail.com

Application User Interface Screenshots

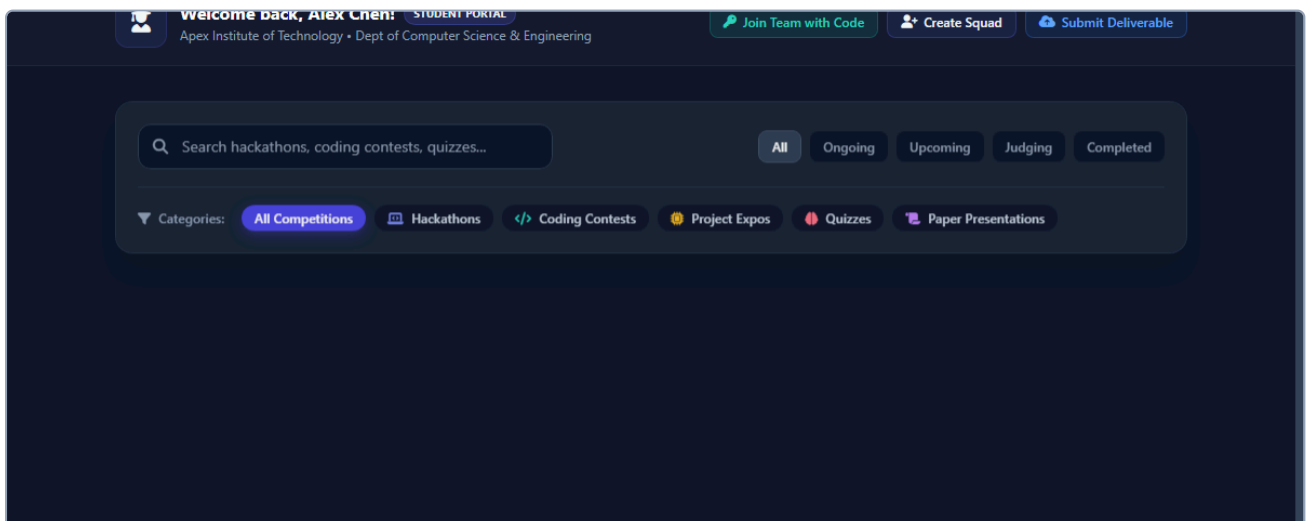
1. Competition Discovery & Catalog



2. CodeCraft In-Browser Interactive Execution Sandbox

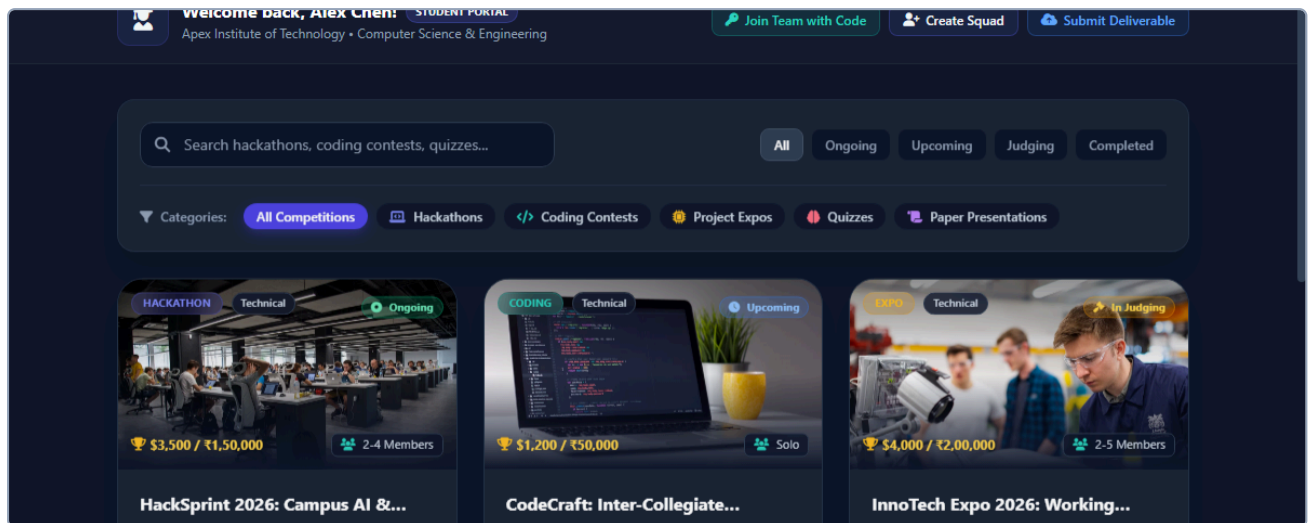


3. BrainByte Quiz Engine & Real-Time Scoring

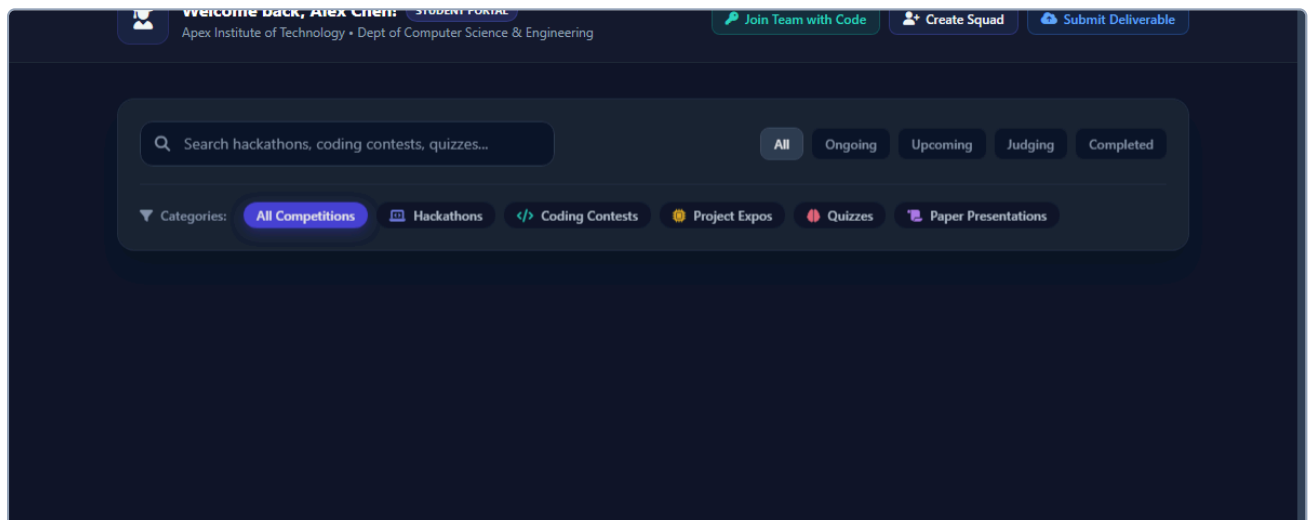


Application User Interface Screenshots (Contd.)

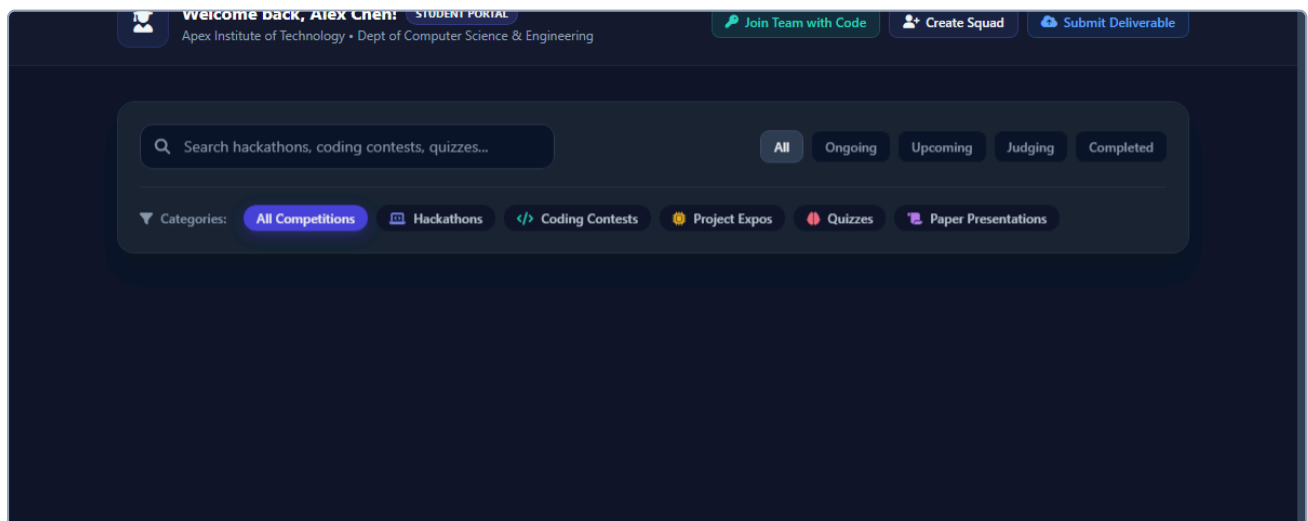
4. Real-Time ELO Leaderboard & Points Matrix



5. Cryptographic Certificate Verification Portal



6. Administrative Oversight & NAAC/NBA Accreditation Dashboard





CERTIFICATE OF COURSE PATHWAY COMPLETION

GUMMINENI KUSHAL

has completed the Course pathway titled
**Web Development Fundamentals: HTML, CSS, and
JavaScript**
which covers
4 courses and **19Hrs** learning hours

CID: LTE/EI/1000

M.F.Febin

Head
L&T EduTech



YALLALA SAI CHARAN
HTML Essentials and Project-Based
Learning

LEARNKONNECT

**Course
Completion**



CERTIFICATE OF COURSE PATHWAY COMPLETION

DANAM LOHITH

has completed the Course pathway titled
**Web Development Fundamentals: HTML, CSS, and
JavaScript**
which covers
4 courses and **19Hrs** learning hours



CID: LTE/EI/1000

A handwritten signature in blue ink, appearing to read "Febin M.F.".

M.F.Febin

Head
L&T EduTech





CERTIFICATE OF COURSE PATHWAY COMPLETION

YEKASI SREENATH REDDY

has completed the Course pathway titled
**Web Development Fundamentals: HTML, CSS, and
JavaScript**
which covers
4 courses and **19Hrs** learning hours

CID: LTE/EI/1000

M.F.Febin

Head
L&T EduTech



CERTIFICATE OF COURSE PATHWAY COMPLETION

BELLAM GURUNAD

has completed the Course pathway titled
**Web Development Fundamentals: HTML, CSS, and
JavaScript**
which covers
4 courses and **19Hrs** learning hours



CID: LTE/EI/1000

A handwritten signature in blue ink, appearing to read "Febin M.F.".

M.F.Febin

Head
L&T EduTech





CERTIFICATE OF COURSE PATHWAY COMPLETION

DUDEKULA SALEEM BASHA

has completed the Course pathway titled
**Full-Stack Web Dev Bootcamp: HTML, CSS, JS, PHP, &
WordPress**
which covers
8 courses and **33Hrs** learning hours



CID: LTE/EI/1000

A handwritten signature in blue ink, appearing to read "Febin M.F.".

M.F.Febin

Head
L&T EduTech

